

Just Enough C++ and git



Prof. Alex Chen

Department of Physics

Washington University in St. Louis

Today

- **Just enough C++:** the small subset you will write every week. It is chosen so that you can also *read* code you did not write.
- **Writing code you can trust:** a few software engineering habits that make your results reproducible.
- **git:** keeps every version of your code, is how you submit homework, and lets you see what your AI agent changed.

C++ is a huge language and we will skip most of it. When you meet a construct you do not recognize, look it up online or ask your agent.

Part I · Just enough C++



The subset you will use

Why C++?

- **Fast.** It compiles to machine code. Most large scientific codes, from LHC analysis software to LIGO detection pipelines, are written in it.
- **Typed.** The compiler checks your program before it runs, so a whole class of bugs is caught before execution.
- **Explicit.** You can see how memory is laid out and what each operation costs.

We will still use Python for plotting and analyzing the data your C++ code produces.

Hello, world



```
#include <iostream>

int main() {
    std::cout << "Hello, world!" << std::endl;
    return 0;
}
```

- `#include` pulls in a library. This one is for console input and output.
- Execution always starts at `main()`.
- `std::` marks names from the standard library.
- Every statement ends with a semicolon. The compiler is not flexible about this.

Compile, then run



C++ is a compiled language: you translate the whole program to machine code first, then execute it.

```
$ g++ -std=c++20 hello.cpp -o hello
$ ./hello
Hello, world!
```

- `-std=c++20`: which edition of the language.
- `-o hello`: name of the executable.

The workflow is always edit → compile → run. If the compile fails, there is nothing to run.

Variables have types

```
int    steps    = 1000;           // whole numbers
double dt       = 0.001;         // floating point (Lecture 1!)
bool    done    = false;         // true / false
std::string label = "run_a";     // text

auto energy = 0.5 * m * v * v;   // compiler deduces: double
```

- A variable's type is fixed when it is declared and never changes.
- Remember Lecture 1: `int` division truncates. `1/2` is `0`; write `1.0/2.0`.
- You can write `auto` when the type is obvious from the right-hand side.

Doing math

```
#include <cmath>

double r      = std::sqrt(x * x + y * y);
double angle  = std::atan2(y, x);
double decay  = std::exp(-t / tau);
double power  = std::pow(base, exponent); // avoid for squares
```

- `<cmath>` has everything: `sqrt`, `sin`, `cos`, `exp`, `log`, `abs`...
- For x^2 , write `x * x`. `std::pow(x, 2)` is slower and no clearer.
- Constants: `std::numbers::pi` from `<numbers>`. Do not type 3.14159.

Making decisions, repeating work

```
if (energy < 0.0) {  
    std::cout << "bound orbit\n";  
} else {  
    std::cout << "escapes\n";  
}
```

```
// the canonical loop  
for (int i = 0; i < n; ++i) {  
    x += v * dt;  
}  
  
while (t < t_end) {  
    t += dt;  
}
```

- Comparison is `==`; assignment is `=`. If you mix them up, the code still compiles, but it is wrong.
- Combine conditions with `&&` (and), `||` (or), `!` (not).
- For loops are fast in C++. Don't be afraid to use them!

`std::vector`: your workhorse

One data structure covers 95% of this course: a dynamic array.

```
#include <vector>

std::vector<double> x(1000, 0.0); // 1000 zeros
x[42] = 3.7;                     // index from 0!
x.push_back(1.0);                // grow by one
std::cout << x.size() << "\n";  // 1001
for (double xi : x) {            // loop over elements
    total += xi;
}
```

You will see old-style arrays (`double x[1000]`) and manual `new/delete` in other people's code. You have no reason to write them.

Functions



Name a computation once; use it everywhere.

```
double kinetic_energy(double m, double v) {  
    return 0.5 * m * v * v;  
}  
  
int main() {  
    double ke = kinetic_energy(1.67e-27, 3.0e5);  
}
```

- The first line says what the function takes (two `double` s) and what it returns (a `double`).
- Functions returning nothing are declared `void`.
- A function must be declared before the point where it is called, so order in the file matters.

Copies vs references



By default, arguments are **copied**. For a million-element vector, that means copying a million numbers on every call.

```
double mean(const std::vector<double>& x) { // no copy
    double sum = 0.0;
    for (double xi : x) sum += xi;
    return sum / x.size();
}
```

- `const&` = read-only reference: no copy, and the compiler will not let you modify it. Use this for anything big.

References that modify

Drop the `const` when the function's job is to change the caller's data.

```
void normalize(std::vector<double>& x) { // no const
    double m = mean(x);
    for (double& xi : x) xi -= m;
}
```

- `&` = reference: work on the caller's object directly. The caller sees the change.
- Default to `const&`. Use a plain `&` only when modifying is the point.

Bundling data: **struct**

```
struct Particle {  
    double x, y;    // position  
    double vx, vy;  // velocity  
    double m;       // mass  
};  
  
Particle p{0.0, 0.0, 1.0, 0.5, 2.0};  
p.x += p.vx * dt;  
std::vector<Particle> cloud(10000); // 10000 particles
```

A **struct** is a group of named values that travel together. That is as object-oriented as this course gets: you will read classes with methods and inheritance in library code, but rarely write them.

Reading compiler errors



- The compiler will reject your program, often. That is good: a compile error is the cheapest bug report you will ever get.
- **Read the first error only.** One missing semicolon can produce 200 lines of errors, and all but the first are usually consequences of it. Fix the first one and recompile.
- The message names the file and line number: `ball.cpp:9:12: error:`
`...` Start there.
- Errors involving standard-library templates can run to pages. Do not try to decode them by hand. Paste the error to your agent and ask what it means.

What we are skipping, on purpose

- Pointers and manual memory management (`new` / `delete`)
- Writing classes, inheritance, virtual functions
- Writing templates (you will happily *use* them: `std::vector<double>` is one)
- Build systems (CMake), until the projects need them

You will run into all of these when reading library code and agent output. We will introduce each one when the course needs it.

The goal: be able to write the subset from today, and read anything.

Part II · Code you can trust



A little software engineering goes a long way

Why bother? You're physicists

- A result you cannot reproduce is not a result. If the code only ran once, on your laptop, in a directory you have since deleted, nobody can check it. That includes you.
- Your most frequent collaborator is yourself three weeks from now, who has forgotten everything and needs the code to explain itself.
- This semester, some of the code in your project will have been written by an agent, not a person. These habits are what let you check that code instead of just trusting it.

Habits that pay for themselves

- **Names mean things.** `dt`, `n_steps`, `total_energy`, not `a`, `tmp`, `foo2`.
- **No magic numbers.** Write `const double G = 6.674e-11;` once, at the top, instead of typing the number into five formulas.
- **Small functions.** If you cannot say what a function does in one sentence, it is two functions.
- **Comments say *why*.** The code already says what. `// leapfrog needs half-step kick first` is useful; `// increment i` is noise.

Test against what you know



Every physics program has cases where you already know the answer: limits, symmetries, conserved quantities, textbook problems.

```
#include <cassert>
#include <cmath>
// free fall from rest:  $x(t) = h - g t^2 / 2$ 
double t_land = drop_time(10.0);
assert(std::abs(t_land - std::sqrt(2.0 * 10.0 / 9.8)) < 1e-3);

// zero velocity means zero kinetic energy
assert(kinetic_energy(5.0, 0.0) == 0.0);
```

You trust a program because it passes checks you designed, not because it looks right or because an AI was confident about it.

Part III · git



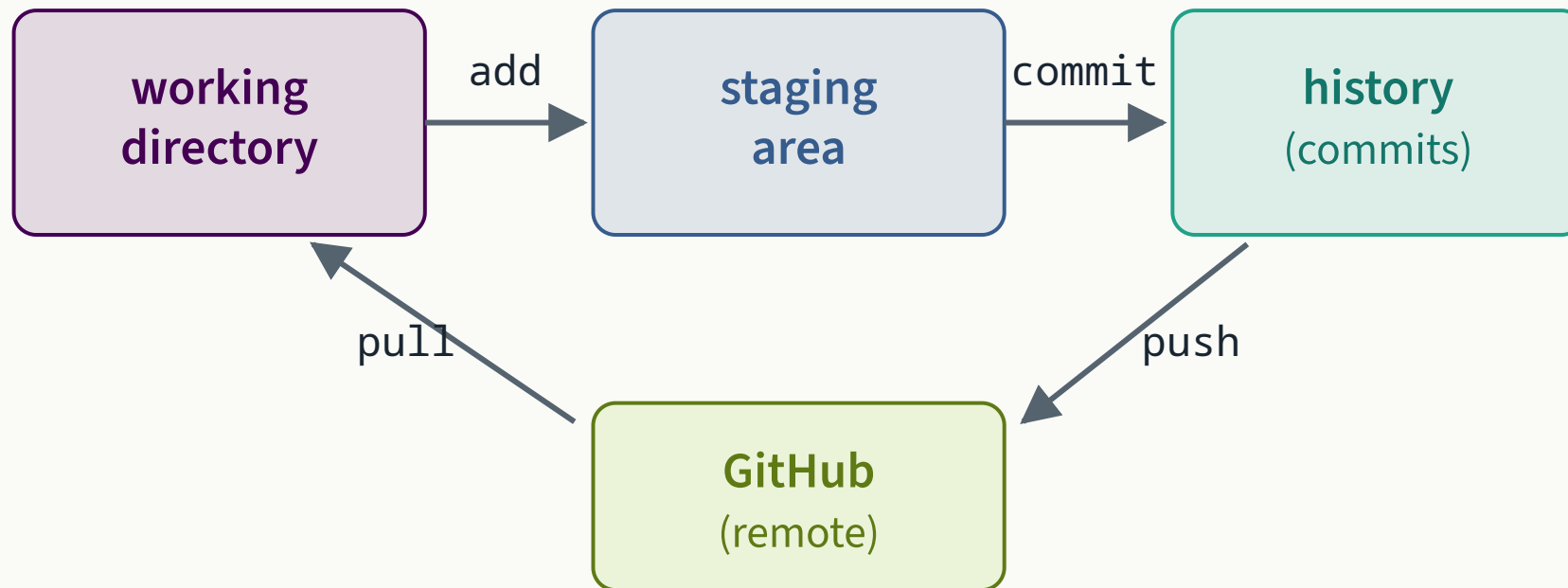
A time machine and a lab notebook for code

The problem

```
$ ls
project.cpp
project_v2.cpp
project_v2_fixed.cpp
project_final.cpp
project_final_REAL.cpp
project_final_REAL_use_this_one.cpp
```

- Which one produced the plot in your report? Nobody knows.
- What changed between v2 and v2_fixed? Nobody knows.
- git replaces this pile with one folder and a complete, searchable history of every version.

The mental model



A **commit** is a snapshot of the whole project, with a message, an author, and a timestamp. Think of it as a lab notebook entry you can go back to.

The daily loop



```
$ git status                # what changed?  
$ git add ball.cpp          # stage what belongs together  
$ git commit -m "Add air resistance to Euler integrator"
```

- Commit whenever the code reaches a state you might want to return to. That usually means several times per work session, not once a week.
- The message should say what changed and why. Not “stuff”, not “asdf”.
- Small, focused commits make the history readable. One idea per commit.

Using the time machine

```
$ git log --oneline           # the notebook's table of contents
9f3e2a1 Add air resistance to Euler integrator
b7c81d0 First working free-fall simulation

$ git diff                   # what have I changed since last commit?
$ git diff b7c81d0           # ...since that older commit?

$ git restore ball.cpp       # discard uncommitted changes
```

When a working program breaks, `git diff` shows exactly what you changed since it last worked. An afternoon of guessing becomes thirty seconds of reading.

GitHub: the shared copy

- git runs entirely on your machine. **GitHub** hosts a copy of the repository everyone can reach.
- `git clone <url>` copies a repository from GitHub onto your machine, history included.
- `git push` sends your new commits to GitHub. `git pull` fetches what changed there.
- Code that has been pushed survives spilled coffee and stolen laptops.

How homework flows in this course



1. **Accept** the assignment link on Canvas. This creates your personal private GitHub repository for that assignment.
2. **Clone** it to your machine.
3. **Work**: edit, compile, run, and commit as you go.
4. **Push**. Whatever is on GitHub at the deadline *is* your submission.

Push early and push often. There is no separate submit button. If it is not on GitHub, it has not been turned in.

git × your agent

- Your AI agent works inside the same repository, and a good one commits its work in small steps, like a colleague would.
- `git diff` is how you *review* that work: read every line that changed before you build on it.
- Commits are checkpoints. If the agent's change broke everything, going back to the last good commit is one command.
- **Rule for the semester: do not commit, and do not keep, agent code you have not read.**

Get set up this week

Before Friday:

1. A C++ compiler. Linux: `g++` from your package manager; macOS: `xcode-select --install`; Windows: WSL (Ubuntu), then `g++`.
2. `git` + a free [GitHub account](#).
3. [VS Code](#) (or your editor of choice).

A setup guide is on Canvas to walk you through installing `g++` and `git`.

And if you have not done the “**AI Literacy for the WashU Scholar**” assignment on Canvas yet: do it ASAP.

Where this goes next

- **Friday:** bring your laptop. We will do a practice session to walk you through some coding tasks, and get you familiar with the homework workflow.
- **Next week:** we will officially start discussing numerical algorithms. Root-finding on Monday.
- References for today:
 - [Pro Git](#), a free book. Chapters 1–2 cover everything from today.
 - learngitbranching.js.org, interactive git practice in the browser.